A Formal Semantics of Mojo 1.2 and Its Ownership Checker

Mark Utting
(with the assistance of Claude; see Acknowledgements)

November 5, 2026
Draft

Abstract

Mojo is a new systems programming language that combines Python-like syntax with value ownership, argument conventions, first-class *origins* and as-soon-as-possible (ASAP) destruction of values. Its memory-safety rules are described in prose and by the behaviour of a closed, rapidly evolving compiler; there is no formal specification. We report on a formalisation in Isabelle/HOL of a subset, micro-Mojo, that is small enough to be tractable but large enough to exhibit the interesting ownership phenomena: a big-step semantics (a clocked definitional interpreter with explicit errors), a flow-sensitive ownership and type checker, an ASAP destructor-insertion pass, and a machine-checked proof that checked programs never fail with an ownership or type error. A formal model proves facts about *itself*, so we also validate it against the real compiler. We built a translator from Mojo source to the Isabelle/HOL syntax that rejects programs outside the subset with an explanation, and used it to generate an Isabelle/HOL theory that evaluates 76 Mojo programs together with the behaviour (output, compile error or run-time error) that Mojo 1.2.0 showed. The model agrees with the compiler on all of them, apart from two that expose a documented gap; differential testing also exposed two further differences that were closed, one by changing the model and one by narrowing the subset.

1 Introduction

Mojo [1] aims to give Python programmers the performance of a systems language. Unlike Python it has no garbage collector: every value has a single owner, and the compiler decides when to destroy it. Unlike Rust, there is no separate borrow-checker vocabulary visible in most code; instead, functions declare how they receive each argument (`read`, `mut`, `var`, `ref`, `out`, `deinit`), references carry a static *origin*, and values are destroyed right after their last use rather than at the end of a scope. These rules decide whether a program compiles, when destructors run and therefore what a program prints, yet they are specified only informally.

This paper describes a formalisation of that core in Isabelle/HOL [3]. Our contributions are:

1. a *region model* of the memory-management proposal for Mojo origins [2], with the design alternatives documented (Section 3);
2. micro-Mojo: abstract syntax, an executable big-step semantics, an ownership and type checker, and an ASAP destructor-insertion pass (Section 4);
3. a machine-checked soundness theorem relating the checker to the semantics (Section 5);
4. a differential-testing pipeline from Mojo source to an Isabelle/HOL theory that is evaluated in batch mode, a suite of 76 programs, and an account of the differences it found (Sections 6 and 7).

The paper is also a case study in a way of working: the entire development, including the proofs, the translator and the test programs, was produced by an AI assistant under the direction of the author, with the proof assistant and the Mojo compiler acting as the independent checkers (see the Acknowledgements).

2 Why formalise, and what makes Mojo hard

Benefits. A formal semantics gives (i) *precision*: writing a rule down forces a decision about every corner the prose leaves open (for example: what happens to a reference into a list when the list is appended to? when exactly is a `var` parameter that the callee never uses destroyed?); (ii) *executability*: the semantics is a function, so example programs can be run and their output compared with expectations before any theorem is proved; (iii) *guarantees*: soundness is a statement about all programs, not those we thought to test; and (iv) a *reference* against which a compiler or an alternative language design can be compared. Because the model is written in a proof assistant, claims such as "the elaboration pass only inserts drops" are checked rather than argued.

Challenges specific to Mojo.

- *No specification.* The documentation describes intent; the compiler defines behaviour, and both are moving (we tested against Mojo 1.2.0; syntax such as `def` versus `fn` and the spelling of conventions has changed between releases). We treat the compiler as a black box oracle.
- *Destruction is observable and implicit.* Destructors (`__deinit__`) can print, so *when* a value is destroyed is part of a program's meaning. Mojo destroys values ASAP, after their last use, not at scope exit, so the semantics needs a last-use analysis, and what counts as a use includes uses of any reference whose origin contains the variable.
- *Many argument conventions.* Each convention changes what the caller may do afterwards (move, mutate, alias) and what the callee may do with the argument; implicit copies are legal for some types and not for others.
- *Origins are first-class.* A returned reference has a declared origin that is a set of parameters; checking a call must compute the origin of its result and extend the lifetimes of the variables in it.
- *Dynamic containers.* A reference into a list is invalidated by operations that change the list's shape but not by writes to its elements; the ownership tree of a list is not static.
- *Flow-sensitive checking.* Initialisation of variables and of individual fields (inside `__init__`) depends on control flow, so the checker is an abstract interpreter with joins at branches and a fixed point at loops.

3 The region model

The starting point is a design sketch by Nick Smith [2], which reinterprets an origin as a first-class, typed compile-time value denoting a *set of variables* a pointer may target. Our theory `Mojo_Region_Model` gives that sketch a formal reading in four parts: (1) a dereference-safety discipline that tracks initialisation of the variables in a region; (2) dynamic containers whose item regions are invalidated precisely when the container's *shape* is mutated; (3) a replacement for Rust's "aliasing XOR mutability" by a disjointness rule on the region *parameters* of a function signature, so that pointers sharing a region parameter may alias freely; and (4) mutability as an

effect declared by a function rather than a property of an origin type. Every choice, and the alternatives not taken, is discussed in comments in the theory. This model is deliberately abstract: it speaks about regions and stores, not about programs. The later development instantiates it, taking the ownership tree of the heap as the owner function.

4 micro-Mojo: syntax, semantics and checker

The subset. micro-Mojo has `Int`, `Bool`, structs and `List[T]`; functions with all six conventions; structs with fields, `@fieldwise_init`, `Copyable`, `__init__(out self,...)`, `__deinit__(deinit self)` and methods; and the statements `var`, `ref` bindings, assignment, `if`, `while`, `return`, `print` and an explicit `drop`. Traits, generics, closures, `raises`, `for`, pointers and overloading are left out (Section 9 discusses extension). Places (`x`, `p.f`, `p[i]`) are a separate syntactic category from expressions, which lets the semantics decide from the callee's convention whether an argument is passed as a location or as a value.

4.1 Abstract Syntax

The syntax is a set of Isabelle/HOL datatypes; the listings in this section are generated from the theory sources, so they cannot drift from them. Types, argument conventions and operators are enumerations:

datatype *ty* = *TInt* | *TBool* | *TStr* | *TNone* | *TStruct name* | *TList ty*
datatype *conv* = *Imm* | *Mut* | *Var* | *Ref* | *Out* | *Deinit*
datatype *binop* = *Add* | *Sub* | *Mul* | *Lt* | *Le* | *Eq* | *And* | *Or*

`TStr` is the type of string literals, which may only be printed, and `TNone` the type of a call that returns nothing. `Imm` is Mojo's default (read-only) convention. `Out` and `Deinit` are the conventions of the `self` parameter of `__init__` and `__deinit__`. Places and expressions are mutually recursive, because an index expression occurs inside a place:

datatype *place* =
  PVar vname — *x*
| *PField place fname* — *p.f*
| *PIndex place expr* — *p*[*i*]
and *expr* =
  EInt int — *42*
| *EBool bool* — *True*
| *EStr string* — *"hi"* (*only as a print argument*)
| *EPlace place* — *x* (*a place used as a value, or as a reference argument*)
| *ETake place* — *p*^ (*transfer*)
| *ECopy place* — *p.copy*()
| *ECall name "expr list"* — *f*(*a*, *b*) *or Res*(*1*) (*call or construction*)
| *EMeth place name "expr list"* — *p.m*(*a*) (*method call*)
| *EBin binop expr expr* — *a* + *b*
| *ENot expr* — *not e*
| *ELen place* — *len*(*p*)
| *EList "expr list"* — [*1*, *2*, *3*]

The comment on each constructor shows the Mojo syntax it represents. A *place* names a location (`x`, `p.f`, `p[i]`). `EPlace p` uses a place as an expression: read as a value it is an implicit copy (legal only for trivial values), passed to a `mut`, `ref` or `imm` parameter it is a reference. `ETake p` is Mojo's transfer `p^`, and `ECopy p` is `p.copy()`. A call `ECall f args` constructs a struct if `f` names one and calls a function otherwise, as in Mojo. Statements are:

datatype *stmt* =
  SPass — *pass*

| *SSeq stmt stmt* — *s1 followed by s2*
| *SVar vname ty "expr option"* — *var x*: $T = e$ *or* *var x*: T
| *SRef vname expr* — *ref r* = *e*
| *SAssign place expr* — $p = e$
| *SExpr expr* — $f(x)$ (*result discarded*)
| *SPrint "expr list"* — *print*(a, b)
| *SIf expr stmt stmt* — *if c*: *s1 else*: *s2*
| *SWhile expr stmt* — *while c*: *s*
| *SReturn "expr option"* — *return e* *or* *return*
| *SDrop place* — inserted by ASAP elaboration

`SDrop` never appears in source programs; it is inserted by the ASAP elaboration described below. Declarations follow:

datatype *ret* = *RNone* | *RVal ty* | *RRef "vname list" ty*

datatype *fundecl* = *Fun*
(*fd-name*: *name*)
(*fd-params*: *"*(*conv* × *vname* × *ty*) *list"*)
(*fd-ret*: *ret*)
(*fd-body*: *stmt*)

datatype *structdecl* = *Struct*
(*sd-name*: *name*)
(*sd-fields*: *"*(*fname* × *ty*) *list"*)
(*sd-copyable*: *bool*)
(*sd-fieldwise*: *bool*)
(*sd-init*: *"fundecl option"*)
(*sd-deinit*: *"stmt option"*)
(*sd-methods*: *"fundecl list"*)

datatype *prog* = *Prog* (*p-structs*: *"structdecl list"*) (*p-funs*: *"fundecl list"*)

A reference-returning function declares its result as `RRef [a, b] T`, Mojo's `-> ref[a, b] T`: the origin is the union of the listed parameters.

4.2 Sugared Syntax

Writing programs as raw constructor applications would be unreadable, so the theory defines notation that mimics Mojo where Isabelle's syntax allows it and uses upper-case keywords elsewhere (in the style of the IMP language). A variable used as a place is marked with a section sign, a field access and an index look like `p.f` and `p[i]`, and a place used as an expression is wrapped in angle brackets. Calls are written with braces around the argument list, and statements are sequenced with a double semicolon. Statements use the keywords `VAR`, `REF`, `::=` (assignment), `+=`, `IF`, `WHILE`, `RETURN`, `PRINT`, `EVAL` (an expression statement) and `DROP`; `p += e` is sugar for assigning $p + e$ to p, as for Mojo's `Int`. The sugar is purely notational: it is parsed to, and printed from, the datatypes above, so every theorem about the syntax applies to sugared programs unchanged.

An example. All examples of the theory come as a Mojo program and its encoding. The first, shown in Figure 1, exercises the conventions, a reference return, a transfer and a list. It uses a resource whose destructor prints, so that the order of destruction is visible in the output, and three small functions. Mojo prints `11`, `drop 2`, `got 11`, `drop 11`, `1`. The value `b` is destroyed straight after the last use of `r`, whose origin is $\{a, b\}$, and before `consume` runs, while `a` is transferred to `consume` and destroyed inside it. The final `append` invalidates `e`, which is harmless because `e` is not used afterwards. The encoding of `main` in the sugared syntax is:

definition *ex1-main* :: *stmt* **where**
"ex1-main =

```
@fieldwise_init
struct Res:
    var id: Int
    def __deinit__(deinit self):
        print("drop", self.id)

def consume(var r: Res):
    print("got", r.id)

def bump(mut r: Res, n: Int):
    r.id += n

def pick(c: Bool, ref a: Res,
         ref b: Res) -> ref[a, b] Res:
    if c:
        return a
    else:
        return b
```

```
def main():
    var a = Res(1)
    var b = Res(2)
    bump(a, 10)
    ref r = pick(True, a, b)
    print(r.id)
    consume(a^)
    var l: List[Int] = [1, 2, 3]
    ref e = l[0]
    print(e)
    l.append(4)

# Output:
# 11
# drop 2
# got 11
# drop 11
# 1
```

Figure 1: The first example: shared declarations (left) and `main` (right), with the output that Mojo 1.2 produces.

$$
\begin{aligned}
&(\mathit{VAR}\ ''a''\ \mathbin{:}\ \mathit{Res} ::= ''\mathit{Res}''\{\![N\ 1]\!\}\ ;;\\
&\mathit{VAR}\ ''b''\ \mathbin{:}\ \mathit{Res} ::= ''\mathit{Res}''\{\![N\ 2]\!\}\ ;;\\
&\mathit{EVAL}\ ''\mathit{bump}''\{\![\langle\S''a''\rangle,\ N\ 10]\!\}\ ;;\\
&\mathit{REF}\ ''r'' ::= ''\mathit{pick}''\{\![TT,\ \langle\S''a''\rangle,\ \langle\S''b''\rangle]\!\}\ ;;\\
&\mathit{PRINT}\ [\langle\S''r''\cdot''\mathit{id}''\rangle]\ ;;\\
&\mathit{EVAL}\ ''\mathit{consume}''\{\![\S''a''\hat{}\,]\!\}\ ;;\\
&\mathit{VAR}\ ''l''\ \mathbin{:}\ \mathit{TList}\ \mathit{TInt} ::= \mathit{LIST}\ [N\ 1,\ N\ 2,\ N\ 3]\ ;;\\
&\mathit{REF}\ ''e'' ::= \langle\S''l''[\![N\ 0]\!]\rangle\ ;;\\
&\mathit{PRINT}\ [\langle\S''e''\rangle]\ ;;\\
&\mathit{EVAL}\ \S''l''\cdot''\mathit{append}''\{\![N\ 4]\!\})''
\end{aligned}
$$

Names are strings, `N 1` is the integer literal, and `TT` is `True`. Note how the call `pick(True, a, b)` gets ⟨§a⟩ for its arguments: the callee's `ref` convention decides, at run time, that a location and not a copy is passed.

4.3 Memory Model

Each variable owns a *root* in a heap; a value is a tree (a struct holds its fields, a list its items), and a location is a path from a root. Mojo's ownership tree is thereby literal, a move transfers a subtree, and destroying a value deinitialises a subtree. Uninitialised and moved-from locations hold a special value, and touching one is the error `ErrUninit`. Every list carries a *generation* number that changes on shape-changing operations; a path through a list records the generation it saw, so following a stale path is `ErrInvalidated`. When a function returns, the roots of its locals are freed and a reference into one is `ErrDangling`.

4.4 Semantics

The semantics is a total, fuel-bounded function in the style of functional big-step semantics [4], with an error monad. Errors are explicit results rather than stuck states, so the theorem "checked programs never reach these errors" is directly expressible, and the semantics can be *run* with `value` and `eval`. Divergence appears as `OutOfFuel`. The result type is `'a res = Ok 'a | Err err | OutOfFuel`.

The semantics is defined for incorrect programs as well as correct ones. A program that breaks Mojo's ownership rules does not get stuck: the interpreter detects the violation at run

time and stops with a specific error. This is what gives the correctness theorem of Section 5 its meaning, since it says that the checker rules these errors out. The errors are:

- `ErrUninit`: reading or writing through an uninitialised or moved-out location, such as a use after `x^`, a variable declared without a value, or a field read before `__init__` has set it.
- `ErrDangling`: following a reference into a root that has been freed, which happens when a function returns a reference to one of its own locals.
- `ErrInvalidated`: following a reference into a list whose shape has changed since the reference was formed, for example after `append` or `pop`.
- `ErrNotCopyable`: copying a value that is neither trivial nor a `Copyable` struct.
- `ErrFieldInit`: an `__init__` that finishes with some field of `out self` still uninitialised.
- `ErrBounds`: an out-of-range list index, including a negative index and `pop` of an empty list. Mojo also reports this at run time, so it is an error of correct programs too.
- `ErrUnbound x`, `ErrUnknown f`, `ErrArity` and `ErrType msg`: use of an undeclared variable, a call of an unknown function or struct, a call with the wrong number of arguments, and an ill-typed operation (for example a `mut` argument that is not a place).

Argument exclusivity, the ban on implicit copies into `var` parameters and moving out of borrowed places are static rules that the interpreter does not check, so it runs such programs anyway (with visibly aliased output, for example); the checker rejects them.

The interpreter is one group of 13 mutually recursive functions. Each takes a fuel n as its first argument after the fixed program P, and each recursive call uses one unit of fuel, which makes termination trivial. Calls make expressions depend on statements (a function body is a statement) and statements on expressions, hence the single group:

function	role
`eval`	expression → *owned* value
`evals`	list of expressions, left to right
`eval_place`	place → path; evaluates and bounds-checks indices, and records the list's generation in the path
`eval_ref`	location-denoting expression (a place, or a call returning a reference) → path
`pass_args`	binds arguments to parameters by convention, giving the callee's initial environment
`call_fun`, `call_meth`	functions, constructors, methods, and the built-in list methods
`invoke`	runs a body in a fresh frame and destroys the frame's locals
`exec`, `exec_block`	statements, and statements in a new block scope
`drop_path`, `drop_paths`	destroy a value: run `__deinit__`, then the remaining fields or list items in order
`cleanup`	destroy and free the owned roots of an environment, newest first

A value is a tree and a location a path, so `eval` returns an owned value while `eval_place` and `eval_ref` return paths into the heap. An environment maps each variable either to an owned root (`Own r`) or to a borrowed path (`Bor q`); this is the only difference between a `var` variable and a `mut` or `ref` parameter.

`pass_args` is where the argument conventions are implemented. For `var` (and `deinit`) the argument is evaluated to an owned value which the callee owns in a fresh root, so `x^` transfers and an implicit copy is checked by the static rules to be of trivial type. For `mut` and `ref` the argument must denote a location, which is borrowed. For `imm` a location holding a trivial value is

copied, since Mojo passes such values by value, other locations are borrowed, and a non-location argument such as a literal becomes a temporary owned by the callee. `call_fun` first looks the name up among the structs: for a struct with an `__init__` it allocates a value whose fields are all uninitialised, passes it as `out self` and afterwards checks that every field is set (`ErrFieldInit`); with `@fieldwise_init` it builds the value from the arguments; otherwise it calls the function. The frame discipline lives in `invoke`, whose equation is generated from the theory:

$$
\begin{array}{l}
\mathit{invoke}\ P\ (\mathit{Suc}\ n)\ \mathit{st}\ \mathit{fd}\ \mathit{env} = \\
\mathit{exec}\ P\ n\ (\mathit{fd\text{-}ret}\ \mathit{fd})\ \mathit{env}\ \mathit{st}\ (\mathit{fd\text{-}body}\ \mathit{fd}) \gg\!= \\
(\lambda(\mathit{oc},\ \mathit{env},\ \mathit{st}). \\
\quad \mathit{cleanup}\ P\ n\ \mathit{st}\ \mathit{env} \gg\!= \\
\quad (\lambda \mathit{st}.\ \mathit{Ok}\ (\mathit{case}\ \mathit{oc}\ \mathit{of}\ \mathit{Normal} \Rightarrow \mathit{RUnit} \mid \mathit{Returned}\ r \Rightarrow r,\ \mathit{st})))
\end{array}
$$

Here $m \gg\!= f$ is bind in the error monad (written with `do` notation in the theory): run the body, then `cleanup` the environment that the body ended with, and return its result. `exec` threads an environment and a heap through a statement and returns whether the statement ended normally or with a `return`; `SVar` allocates a fresh root and extends the environment, `SAssign` destroys the old value of the place before storing the new one, and `SDrop p` calls `drop_path`. `drop_path` runs the struct's `__deinit__` with `self` bound to the location and then destroys the fields in order; this is the code that gives destruction its observable effect, since a destructor can `print`. A program is run from `main`:

$$
\begin{array}{l}
\mathit{run}\ \mathit{fuel}\ P = \\
(\mathit{case}\ \mathit{find\text{-}fun}\ P\ ''\mathit{main}''\ \mathit{of}\ \mathit{None} \Rightarrow \mathit{Err}\ (\mathit{ErrUnknown}\ ''\mathit{main}'') \\
\ \mid \mathit{Some}\ \mathit{fd} \Rightarrow \\
\quad \mathit{invoke}\ P\ \mathit{fuel}\ \mathit{init\text{-}state}\ \mathit{fd}\ [] \gg\!= (\lambda(\mathit{uu},\ \mathit{st}).\ \mathit{Ok}\ (\mathit{out\text{-}lines}\ \mathit{st})))
\end{array}
$$

and the result is the list of printed lines (a line is a list of values). Output printed before an error is not part of the result, because the error monad does not carry the state.

4.5 ASAP Elaboration

To keep the dynamic semantics free of last-use analysis, a static pass `elaborate` inserts explicit `DROP` statements: after each statement, the owned variables it used that are not live afterwards are dropped, branches drop variables that are live before the `if` but not within the branch, and loop bodies are elaborated against the loop's live set. A use of a reference counts as a use of every variable in its origin. Destroyed values run their `__deinit__` and then the destructors of their remaining fields and list items, in order. Programs without `DROP`s still have a C++-like meaning (owned locals are destroyed in reverse order at scope end), which never fires after elaboration.

4.6 Ownership Checker

`check_prog` is a flow-sensitive abstract interpretation of each function body, with an abstract state recording (i) which owned variables are definitely initialised, (ii) which reference bindings may have been invalidated by a shape change of a list they point into, and (iii) inside `__init__`, which fields of `out self` are initialised. It also checks types, that implicit copies are only of trivial values, that arguments are exclusive (comparing the origins of borrowed arguments, the disjointness rule of the region model), that immutable arguments are not mutated, and that a returned reference comes from its declared origin. Some restrictions are deliberate simplifications that make the checker *stricter* than Mojo: variable names must be distinct within a function, static paths stop at the first index (so `l[i]` and `l[j]` always overlap), and analysis is at statement rather than sub-expression granularity.

5 Correctness results

The main result of the formalisation is a machine-checked guarantee about the ownership checker. It is the formal counterpart of the promise that Mojo programmers rely on every day: *if the compiler accepts your program, it will not misuse memory.*

> **Theorem (safety of checked programs).** If the ownership checker accepts a micro-Mojo program, then running it, with any amount of fuel, never fails with a use of a moved or uninitialised value, a use of an invalidated or dangling reference, a copy of a non-copyable value, a struct left partly initialised by its `__init__`, or any type, scoping or arity error. The only error it can report is an out-of-range list index, which Mojo also reports at run time.

In Isabelle/HOL the theorem and its corollary read (slightly abridged):

```
theorem run_safe:
  assumes "wf_prog P"
  shows "safe (run fuel P)"

corollary run_no_ownership_error:
  assumes "wf_prog P" and "run fuel P = Err e"
  shows "e = ErrBounds" and "~ ownership_err e"
```

What `wf_prog` means. `wf_prog P` is defined as `check_prog P = []`: the checker of Section 4 returns the list of errors in a program, and a *well-formed* program is one for which the list is empty. In Mojo terms, well-formed means "the compiler accepts it". It covers everything the checker does: types, definite initialisation and moves, invalidation of references, exclusivity of arguments, the origins of returned references, field initialisation in `__init__`, and structural conditions such as distinct names and a parameterless `main`. Because the hypothesis is only the checker's verdict, the theorem is about *all* accepted programs, not about the examples we tried, and it quantifies over every possible execution (the semantics is deterministic) and every fuel bound.

What `safe` means. `safe r` says that the outcome `r` of running a program is not an error other than a bounds error: `safe r = (`$\forall e.\ r = Err\ e \longrightarrow e = ErrBounds$`)`. So the program either finishes, or runs out of fuel (it is still running; there is no bug yet), or stops at an out-of-range index. `ownership_err` collects the errors that correspond to ownership mistakes (`ErrUninit`, `ErrDangling`, `ErrInvalidated`, `ErrNotCopyable`, `ErrFieldInit`), and the corollary says none of them can occur. Since the interpreter of Section 4 raises exactly these errors for programs that break the rules, the theorem says that the checker catches every such program *before* it runs.

Why it matters. Without such a theorem, "the checker accepts it" is a hope. With it, every class of memory error in Table 1 is ruled out for every accepted program, including those whose bug depends on a particular control-flow path, a loop that runs several times, a reference that outlives a function, or a list that is appended to while a reference into it is live. The result is stronger than ownership safety alone: because the checker also checks types, and excluding uninitialised reads needs well-typed values anyway, type and arity errors are excluded too. The one error left, an out-of-range index, is not an ownership error, and Mojo reports it at run time as well.

Programs the checker rejects. Figure 2 shows one small Mojo program for each ownership error. Each is rejected by Mojo and by our checker; run without checking, the interpreter ends in the error named above it. These are exactly the programs on which the theorem is silent, and it is silent on them because the checker refuses them.

Run-time error	The Mojo mistake it models	Excluded?
`ErrUninit`	use after move; use of an unset variable or field	yes
`ErrInvalidated`	using a reference after `append`/`pop` on the list	yes
`ErrDangling`	returning a reference to a local	yes
`ErrNotCopyable`	implicit or explicit copy of a non-`Copyable` value	yes
`ErrFieldInit`	`__init__` that leaves a field unset	yes
`ErrUnbound`, `ErrUnknown`, `ErrArity`, `ErrType`	undeclared names, wrong argument count, ill-typed operations	yes
`ErrBounds`	list index out of range, `pop` of an empty list	**no** (Mojo also fails at run time)

Table 1: What the safety theorem rules out for checked programs.

```
# ErrUninit
var a = Res(1)
consume(a^)
print(a.id) # a was moved

# ErrInvalidated
var l: List[Int] = [1, 2, 3]
ref e = l[0]
l.append(4)
print(e) # e invalidated

# ErrNotCopyable
var a = Res(1) # Res is not Copyable
var b = a.copy()
```

```
# ErrDangling
def dangle(ref a: Int) -> ref[a] Int:
    var local = 5
    return local # escapes

# ErrFieldInit
struct Point:
    var x: Int
    var y: Int
    def __init__(out self, a: Int):
        self.x = a # y never set
```

Figure 2: Programs rejected by the checker, and the run-time error each would cause.

How it is proved. The central invariant `inv` relates a static environment Γ and an abstract state σ to a run-time environment and heap: an owned variable that is definitely initialised holds a well-typed value; a borrowed parameter denotes a well-typed value (except the fields of `out self` not yet initialised); and a reference binding denotes a path below its base variable which is well-typed unless the reference is flagged as invalidated. Soundness is proved by a single induction over the fuel, because expressions, statements and calls are mutually recursive: if the checker accepts a statement in a state satisfying `inv`, the interpreter's result is not an error (apart from a bounds error) and the final state satisfies `inv` for the checker's output state. The same statement is proved for expressions and statements (`expr_sound_calls`, `stmt_sound_calls`); `run_safe` is the instance for `main`.

The proofs occupy about 8 700 lines and 344 lemmas (Table 2), with the call case alone taking more than half.

The proof found bugs in the checker. A definition that merely looks right can be wrong in corners; a soundness proof cannot be completed until it is right. Proving the case of calls exposed several holes in the checker, which were fixed: places still in use while a later argument or index is evaluated; the origin of a reference returned by a call; `ref[ns]` origins that are not `mut` or `ref` parameters; reference-returning calls used as values of non-trivial type; an early `return` in an initialiser; the receiver parameters of initialisers and methods; duplicate field names; name clashes between functions and structs; and a `main` with parameters. Each was a hole that the proof could not close until the checker was strengthened.

Theory	Content	Lines	Lemmas
`Mojo_Region_Model`	region model	1 185	27
`Micro_Mojo_Syntax`	syntax, sugar, examples	584	–
`Micro_Mojo_Semantics`	interpreter, examples	805	15
`Micro_Mojo_Static`	checker, elaboration	1 177	10
`Micro_Mojo_Soundness`	expressions without calls	1 169	67
`Micro_Mojo_Soundness_Stmt`	statements	2 048	71
`Micro_Mojo_Soundness_Calls`	calls, whole programs	5 469	206

Table 2: Size of the development (lines of theory text; lemmas, theorems and corollaries).

What the theorem does not say.

- It says the *model's* checker is sound for the *model's* semantics. It does not say the model resembles Mojo: a checker that rejected every program would satisfy it. This gap is the motivation for the next section.
- It covers programs as the checker accepts them, with or without explicit `DROP` statements. Elaboration (Section 4.5) is proved to insert only `DROP`s (`elab_only_inserts_drops`: after stripping `DROP`s the program is unchanged), but we have not proved that an elaborated program is again accepted by the checker. That is validated by the differential tests, which run elaborated programs, and by the examples, not by a proof.
- Termination is not claimed: fuel is exhausted, not diagnosed.
- Out-of-range indices are not excluded, as above.

6 Differential testing

Idea. To validate fidelity, we generate Mojo programs, run them with the real compiler to obtain the observed outcome, translate the same programs to micro-Mojo, and ask Isabelle/HOL to evaluate them. Each test has one of three expected outcomes: the program prints a given list of lines; it is rejected at compile time; or it fails at run time. The Isabelle/HOL predicates are, respectively, that the elaborated program run by the interpreter produces that output, that `check_prog` reports an error, and that the run ends with an error. Output is rendered as text (integers in decimal, booleans as `True`/`False`, arguments space-separated) so that it can be compared with the compiler's standard output.

Pipeline.

$$\texttt{prog.mojo} \xrightarrow{\text{translator}} \text{Isabelle/HOL term of type } \texttt{prog}$$

$$\texttt{prog.mojo} \xrightarrow{\text{mojo build, run}} \texttt{prog.expected.json}$$

$$\text{both} \xrightarrow{\text{generator}} \texttt{Micro_Mojo_Suite.thy} \xrightarrow{\text{isabelle build}} \text{pass/fail}$$

The translator is a hand-written tokenizer (with Python-style indentation), a recursive-descent parser and a translator with type inference, in about 1 500 lines of Python. Its most important property is what it *refuses*. Any construct outside the subset is rejected with a message that names the feature and says why it is not supported, for example:

```
suite/rejected/a09_short_circuit.mojo:7:18: unsupported Mojo feature: right
operand of 'and' with calls or indexing
    Mojo short-circuits 'and'/'or' but the micro-Mojo semantics evaluates both
    operands; use nested 'if' statements when the right operand has side
    effects or can fail
```

A silent mistranslation would invalidate the whole experiment, so the translator is conservative: it also rejects integer literals beyond 10^6 (see below), and it is unit-tested on about forty rejection cases. We further proved in Isabelle/HOL that the translator's output for the example programs of the syntax theory is *equal* to the hand-written encodings, so the two independent encodings cross-check each other.

Recorded results. The driver records, for each test, the compiler's status, standard output and diagnostics, and the compiler version, in a JSON file. The generated theory therefore needs no Mojo installation to be re-evaluated: it is one `lemma` per program, proved by `eval`, and the whole suite builds as an ordinary Isabelle/HOL session in about two minutes.

The test programs. There are 11 programs from the original syntax theory and 65 new ones designed to probe likely weak spots: timing of destruction (14), moves (including in branches and loops), implicit and explicit copies, reference invalidation, lists, `mut`/`var` parameters, methods and initialisers, and arithmetic and control flow. Mojo accepted and ran 53 of the 76 programs, rejected 20 at compile time and aborted 3 at run time (out-of-range index, pop of an empty list, negative index). Compile-error tests are deliberately written so that only one rule is violated, and we recorded Mojo's diagnostics to check that the reason is the intended one (use of an uninitialised value, invalidated interior reference, aliasing a mutably borrowed value, and so on). We note one limitation: the Isabelle/HOL test asserts only *that* the checker rejects the program, not that it does so for the same reason.

The 11 original examples had expected outputs written by hand *before* the compiler was run; Mojo agreed with every one.

An example. Program `d13` has a struct whose destructor prints and which contains three fields with destructors of their own:

```
struct Outer:
    var a: Inner
    var b: Inner
    var c: Inner
    def __deinit__(deinit self):
        print("outer start")
        print("a is", self.a.id)
        print("outer mid")
        print("b is", self.b.id)
        print("outer end")
```

Mojo prints `drop inner 3` *before* `outer start` (field `c` is never used, so it is destroyed on entry), then destroys `a` immediately after its last use and `b` likewise. A scope-based model, or one that runs the body and then destroys the fields in order, gets a different trace.

7 What differential testing found

Of the 65 new programs, 61 agreed on the first evaluation. The four that did not revealed three kinds of gap, plus a tooling issue.

Unused `var` parameters. The model dropped an owned parameter only after its last use, so one that was never used lived until the callee returned. Mojo destroys it on entry. The fix was a small change to the elaboration pass: the body starts with drops of the owned parameters that are not live on entry. Because the soundness proofs are about the checker and interpreter, not the particular elaboration, the proofs were unaffected, which we confirmed by rebuilding.

Difference	Mojo 1.2.0	Resolution
Unused `var` parameter (d09, d14)	destroyed when the callee is entered	**model fixed:** elaboration drops it on entry; proofs unaffected
Fields in `__deinit__` (d05, d13)	each field destroyed ASAP; unused ones before the body	**documented divergence;** test asserts the mismatch
`and`/`or` (a09)	short-circuit	translator **rejects** impure right operands
Variable reuse in disjoint scopes (a08)	allowed	translator **rejects**; checker requires unique names

Table 3: Differences between the model and Mojo 1.2.0 found by the suite.

(The flip side is that the soundness theorem says nothing about the elaboration, as discussed in Section 5.)

Field-wise destruction in `__deinit__`. In the model, `__deinit__` runs as a body over a borrowed `self` and afterwards the remaining fields are destroyed in order. Mojo instead treats the fields of `deinit self` as separately owned, and applies the ASAP rule to each. Matching this would require tracking initialisation per field in the checker's abstract state, a drop rule for fields of `deinit self`, an elaboration over field places rather than variables, and changes to the invariant used throughout the soundness proofs. We judged the cost not worth it for a divergence that is visible only when a struct with printing field destructors also has a printing `__deinit__`, and recorded it instead. Each known divergence is a file entry with a reason, and the generated theory asserts *the negation* of the test for it. The build stays green today, and becomes red as soon as someone fixes the semantics, at which point the entry is deleted and the test becomes an ordinary one. This turns a known gap into a regression test for the future fix.

Short-circuit `and`/`or`. The interpreter evaluates both operands. Making it short-circuit would change the evaluation function and the induction in every soundness case that involves boolean operators. Since the difference is observable only when the right-hand operand has effects (a call that prints, or an indexing that may fail), we narrowed the subset: the translator rejects such programs and explains why. The rejected example is kept in a directory of its own, and a unit test checks that it stays rejected, so that the restriction cannot silently disappear.

Variable names. Mojo allows the same `var` name to be declared in disjoint scopes. The model's checker requires unique names in a function (this simplifies the soundness invariant, since the environment can be treated as a finite map from names to kinds). Again the translator rejects such programs.

A tooling observation. Evaluating integer arithmetic around 10^9 with `eval` hung on the (32-bit-pointer) Poly/ML platform we used, even for plain natural-number arithmetic. The translator therefore limits literals to 10^6 in absolute value. We mention this because a differential test harness has its own trusted base, and its failures can look like failures of the semantics.

Agreement on rejection. All 20 compile-error programs were rejected by the checker, and all three run-time errors were reproduced as out-of-range accesses. In particular the checker

rejected partial moves, the use of references after a shape-changing mutation, conflicting borrows of the same origin, and `__init__` that does not initialise every field, each as Mojo did. Since the model's checker is intentionally stricter than Mojo in places, the converse direction (programs Mojo accepts but the model rejects) is the more informative one: the cases we found there are the restrictions of Section 4, which are surfaced as translator rejections where they matter.

8 Threats to validity

- The suite was designed by the same agent that wrote the semantics, so it may share blind spots. The first-pass agreement on the hand-predicted examples is encouraging but not independent evidence. Random program generation à la Csmith [20] would complement it.
- Tests assert the *status* of a compile error, not its cause.
- The suite targets one compiler version; Mojo changes quickly.
- The theorem is relative to the model; each translator rejection and each recorded divergence is part of the model's stated scope and should be read alongside it.
- Values are small (decimal integers below 10^6); arithmetic overflow and floating point are not modelled.

9 Related work

Rust. The closest bodies of work formalise Rust's ownership. RustBelt [5] proves semantic soundness of a core calculus and its unsafe libraries in Iris; Oxide [6] gives a syntactic type system for borrowing based on "provenances" (regions of loans); Featherweight Rust [7] models lifetimes and borrowing in a small calculus; Stacked Borrows [8] and Tree Borrows [9] define aliasing models for unsafe code and are validated with the Miri interpreter. Non-lexical lifetimes [10] are the analogue of our flow-sensitive checker. Our work differs in the object language, which has argument conventions, ASAP destruction and origins as sets of variables, and in method: a single proof assistant development containing the semantics, the checker and a compiler-validated test suite. Gao and Parreaux's InvalML [11], raised in the Mojo design discussion, is a closer model of invalidation.

Regions. Tofte and Talpin's region calculus [12] and Cyclone's regions [13] are the origin of the idea that a pointer's type names the memory it may refer to. Mojo's origins differ in that they are sets of *variables* and are first-class values.

Ownership in other languages. Swift's `borrowing` and `consuming` conventions [14] are similar to Mojo's argument conventions; C++ destructors give scope-based rather than ASAP destruction.

Mechanised language semantics and testing. CompCert [15], the WebAssembly specification [16], JSCert [17] and the executable C semantics [18] demonstrate the value of executable formal semantics tested against implementations. Differential testing in general goes back to McKeeman [19] and was made famous for compilers by Csmith [20]. Our setting differs in that the oracle is a compiler for a language whose semantics is not otherwise specified, and the object under test is the formal semantics itself. The use of a clocked definitional interpreter follows Owens et al. [4].

10 Conclusion and future work

We have shown that a core of Mojo's ownership discipline, including ASAP destruction, argument conventions, origins and invalidation of references into dynamic containers, can be given an executable semantics and a checker whose soundness is proved in Isabelle/HOL, and that a pipeline from Mojo source to an Isabelle/HOL theory lets the compiler itself be used as an oracle in batch mode. The exercise produced concrete findings: a model bug fixed (unused `var` parameters), a gap documented and kept as a regression test (field-wise `__deinit__`), and two restrictions of the subset made explicit in the translator. We found the combination valuable: the proof tells us that the checker implies the absence of ownership errors, and the testing tells us how far the model is from Mojo.

Future work: (i) field-wise destruction in `__deinit__`; (ii) short-circuit operators and block scoping with shadowing; (iii) pointers with origins and `for` loops over lists; (iv) traits, generics and `raises`; (v) random generation of programs, including mutation of existing tests, and tracking the causes of compile errors; (vi) re-running the suite on newer Mojo versions to track the language's evolution.

Acknowledgements

The author set the goals, took the design decisions that were put to them (including the choice of a clocked functional big-step semantics and of an explicit `drop` elaboration), directed each phase (region model, subset proposal, syntax, semantics, checker, soundness, differential testing, larger suite), asked for the compiler to be used once it was found to be installed through `uv`, and decided what to commit and what to leave as a documented divergence.

The development itself was carried out by Claude (Anthropic), working through Claude Code, with the Isabelle PIDE MCP server for interactive proof development; the final phases were done with Claude Sonnet 5.5. Claude wrote the region theory and its commentary, the micro-Mojo syntax, semantics, checker and elaboration, the soundness proofs, the Python translator and test driver, the unit tests, the 76 Mojo test programs, the build configuration, the READMEs, the analysis of each test disagreement, the small fix to the elaboration pass, and a first draft of this paper. In the testing phase, Claude first wrote the expected output for each example by hand from the semantics and only then ran the compiler, which agreed. The validity of the formal results rests on the Isabelle kernel and of the empirical results on the Mojo compiler, both independent of the assistant that produced the artefacts; the statements of the definitions and theorems, the choice of subset and this paper's claims should still be read critically by a human, and the bibliography below has been written from memory of the literature and needs checking against the sources.

References

[1] Modular. *Mojo Manual: Ownership and value lifecycle.* `https://docs.modular.com/mojo/manual/`. Accessed 2026.

[2] N. Smith. *An alternative model for "lifetimes" in Mojo.* GitHub gist and discussion. `https://gist.github.com/nmsmith/cdaa94aa74e8e0611221e65db8e41f7b`.

[3] T. Nipkow, L. C. Paulson, M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic.* LNCS 2283, Springer, 2002.

[4] S. Owens, M. O. Myreen, R. Kumar, Y. K. Tan. Functional big-step semantics. ESOP 2016.

[5] R. Jung, J.-H. Jourdan, R. Krebbers, D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. POPL 2018.

[6] A. Weiss, D. Patterson, N. D. Matsakis, A. Ahmed. Oxide: The essence of Rust. arXiv:1903.00982, 2019.

[7] D. J. Pearce. A lightweight formalism for reference lifetimes and borrowing in Rust. ACM TOPLAS 43(1), 2021.

[8] R. Jung, H.-H. Dang, J. Kang, D. Dreyer. Stacked Borrows: An aliasing model for Rust. POPL 2020.

[9] N. Villani, J. Hostert, D. Dreyer, R. Jung. Tree Borrows. PLDI 2025.

[10] N. D. Matsakis. Rust RFC 2094: Non-lexical lifetimes. 2017.

[11] Gao and Parreaux. InvalML. OOPSLA 2025. (Reference as cited in the design discussion [2]; details to be verified.)

[12] M. Tofte, J.-P. Talpin. Region-based memory management. Information and Computation 132(2), 1997.

[13] D. Grossman, G. Morrisett, T. Jim, M. Hicks, Y. Wang, J. Cheney. Region-based memory management in Cyclone. PLDI 2002.

[14] Swift Evolution SE-0377: `borrowing` and `consuming` parameter ownership modifiers.

[15] X. Leroy. Formal verification of a realistic compiler. CACM 52(7), 2009.

[16] C. Watt. Mechanising and verifying the WebAssembly specification. CPP 2018.

[17] M. Bodin, A. Charguéraud, D. Filaretti, P. Gardner, S. Maffeis, D. Naudziuniene, A. Schmitt, G. Smith. A trusted mechanised JavaScript specification. POPL 2014.

[18] C. Ellison, G. Rosu. An executable formal semantics of C with applications. POPL 2012.

[19] W. M. McKeeman. Differential testing for software. Digital Technical Journal 10(1), 1998.

[20] X. Yang, Y. Chen, E. Eide, J. Regehr. Finding and understanding bugs in C compilers. PLDI 2011.